Practica 2: Integración numérica distribuida mediante Java RMI

Computación Distribuida — Marcos García Blanco

1. Solución

La solución diseñada se realiza en cinco ficheros distintos. *MonteCarloInterface*, *MonteCarloServer*, *MonteCarloImpl*, *MonteCarloClient* e *HiloCliente*.

2. Instrucciones para compilación y ejecución

Para la compilación situándonos en la carpeta con todos los archivos *.java* y se ejecuta el comando *javac *.java*. Para la ejecución se necesitan varias terminales, una por cada servidor y otra para el cliente. Cada servidor se lanza con *java MonteCarloServer <Puerto> [IP]*. El número del puerto es un argumento obligatorio y la IP se le puede pasar o no, si se deja vacío toma automáticamente la IP como *localhost*. Para el cliente se lanza con *java MontecarloClient <N> <tamBloque> <ip1:puerto1> [ip2:puerto2] ... [ipN:puertoN]*, en esta el primer argumento es el número de puntos, el segundo el tamaño de cada bloque y luego un argumento que indica la IP y el puerto de un servidor.

Un ejemplo de ejecución de un servidor en localhost es: *java MonteCarloServer 1099* y un ejemplo de ejecución de un cliente es: *java MonteCarloClient 10000000 100000 localhost:1099 localhost:1100.*

3. Resultados

Para las pruebas se utilizará un N = 100.000.000 y un tamaño de bloque de 100.000, estos valores se mantendrán siempre constantes sin importar el número de servidores. Recordemos que para calcular el Speedup se usa la formula: $S_p = \frac{T_1}{T_p}$ y para calcular la eficiencia se usa $E_p = \frac{S_p}{p}$. Los valores obtenidos se recogen en la siguiente tabla:

Servidores	Tiempo (s)	Speedup	Eficiencia
1	1.533432592	1	1
2	0.753649224	2.034676801	1,0173384005
4	0.375319023	4.085677778	1,0214194445
8	0.211317722	7.256526227	0,907065778375

Los resultados cuadran con lo esperado. El tiempo se reduce casi a la mitad cada vez que se duplican los servidores: el *speedup* es 2,03 con 2 servidores, 4,09 con 4 y 7,26 con 8. La eficiencia es 1,02, 1,02 y 0,91. La eficiencia mide qué tan cerca estamos de que todos los servidores se usen al 100% (con p servidores se debería ir p veces más rápido), este caso ideal sería 1. Con 2 y 4 servidores sale casi 1 (un poco más por pequeñas variaciones al medir). Con 8 baja a 0,91, porque el programa tarda tan poco (0,21 s) que los tiempos que no se pueden repartir, como arrancar los hilos y conectar con los servidores, ya se notan. Aun así, 0,91 es un resultado muy bueno, y se ve que añadir servidores compensa.

4. Cuestiones

1. ¿Por qué la aproximación mejora al aumentar el número de puntos generados?

La integral de e^(−x²) entre 0 y 1 es el área bajo la curva. Como los puntos se lanzan al azar dentro de un cuadrado de área 1, la proporción de puntos que caen bajo la curva (M/N) es una estimación de esa área. Con pocos puntos la suerte influye mucho, y con muchos se compensa, por lo que la proporción se acerca cada vez más al área real. Aun así, es una mejora probable y no está garantizada en cada ejecución concreta.

2. ¿Por qué es necesario utilizar hilos en el cliente para que las llamadas RMI se realicen en paralelo?

Cuando el cliente llama a un servidor se queda esperando la respuesta y no puede hacer nada más debido a que la llamada de RMI es bloqueante. Con un hilo por servidor, cada hilo espera a su servidor en paralelo, de esta forma todos calculan a la vez. Sin hilos, el cliente tendría que llamar a los servidores uno detrás de otro, esperando a que cada uno termine antes de poder llamar al siguiente por lo que el tiempo no mejoraría y sería como tener un único servidor.

3. ¿Qué ocurriría con el tiempo de ejecución si el cliente realizase las llamadas a los servidores secuencialmente?

Tardaría prácticamente lo mismo que si solo se usase un único servidor, ya que como he explicado en la pregunta anterior cada servidor tendría que esperar a que acabase el anterior. Por tanto, añadir nuevos servidores no serviría de nada.

4. ¿Es de esperar que utilizar el doble de servidores reduzca exactamente a la mitad el tiempo de ejecución? Justifique la respuesta.

No exactamente, hay partes que no se pueden repartir como el coste de crear los hilos, conectar con los servidores o sumar los resultados, además de que las comunicaciones también cuestan tiempo. Por eso el tiempo se acerca a la mitad, pero no suele llegar justo.

5. ¿Qué parte del tiempo de ejecución puede corresponder a las comunicaciones RMI?

Cada vez que el cliente llama a un servidor tiene que mandarle el número de puntos, esperar y recibir el resultado, esto tarda un poco y se hace siempre, se calculen pocos o muchos puntos. Con muchísimos puntos casi no se nota, porque el tiempo se va en calcular. Pero en el reparto dinámico hay una llamada por cada bloque, así que son muchas más y las comunicaciones pesan más, sobre todo si los bloques son pequeños. Por tanto, si hay muchos servidores, pocos puntos o los bloques son muy pequeños se puede acabar notando este tiempo, aunque sea mínimo.

6. ¿Qué ocurriría si uno de los servidores dejase de responder durante la ejecución?

Hay dos casos, si el servidor se cae, la llamada da un error y ese hilo termina. El cliente no cuenta los puntos de ese servidor, así que la integral tiene un error debido a que muchos puntos que habrían caído bajo la curva no se cuentan, pero se sigue dividiendo entre el total de puntos inicial. En el caso del reparto dinámico el problema sería ligeramente menor ya que solo se perdería el bloque en el que se cae el servidor.

Por otro lado, si el servidor se queda colgado sin cerrarse, el cliente se queda esperando para siempre, porque RMI no tiene tiempo límite por defecto y el join() nunca termina.

7. ¿Cómo podría modificarse el cliente para detectar que un servidor no responde y evitar que la aplicación quede bloqueada indefinidamente?

Si el servidor se cae, el hilo recibe una excepción, así que se podría añadir un atributo error que se active al capturarla. Después del join(), el cliente consultaría ese atributo para saber qué servidor ha fallado y avisar de que el resultado no es completo. Si el servidor se queda colgado, no hay excepción, así que habría que poner un tiempo máximo de espera con join(tiempo) en lugar de join(). Si pasado ese tiempo el hilo sigue vivo (isAlive()), se considera que el servidor ha fallado. Una vez detectado el fallo el resultado se podría corregir, una opción es restárselo al total y dividir entre los puntos realmente calculados, en vez de entre N, otra es devolver ese bloque a los puntos pendientes para que otro servidor lo calcule. Así el resultado seguiría siendo correcto aunque falle un servidor.

8. ¿Qué problemas pueden aparecer si los servidores tienen capacidades de cálculo muy diferentes?

Con el reparto estático, el servidor más lento tarda más, y los demás terminan antes y se quedan esperando sin hacer nada. El tiempo total lo marca el más lento, y se desaprovechan los servidores rápidos. Por tanto, el reparto dinámico lo mejora porque los servidores rápidos piden más bloques y los lentos menos.

9. ¿Por qué un reparto perfectamente equilibrado en número de puntos no garantiza necesariamente un reparto perfectamente equilibrado en tiempo de ejecución?

Porque repartir los mismos puntos significa dar el mismo trabajo, pero no garantiza la misma velocidad ya que un servidor puede ser más potente que otro, estar haciendo otras tareas a la vez, tener peor conexión con el cliente... Entonces, con los mismos puntos, tarda más o menos.